Shai Goldberg-Kellman
Gabe Small

PS2
COSI 180a

We asked for an extension but we have not heard back so I am uploading this just in case. If we will get the extension please disregard this

Q1. Recall the interval scheduling problem discussed in class, which is an example of an activity-selection problem. Suppose that instead of always selecting the first activity to finish, you instead select the last activity to start that is compatible with all previously selected activities. Describe how this approach is a greedy algorithm and prove that it yields an optimal solution.

This is just the same algorithm, but working from the end instead of the beginning.

Let $i_k, \dots, i_1$ be the jobs selected by the algorithm (arranged by starting time from the end where i_1 is the last job, i_2 is the second to last and so on)
Let $j_m, \dots, j_1$ be the jobs in an optimal solution

Assume that jobs 1-r are the same between the algorithm and the optimal solution.
The greedy solution picks job i_{r+1} and the optimal solution pick job j_{r+1}.

j_{r+1} exists because $m > k$
i_{r+1} cannot start any earlier than j_{r+1}, otherwise the algorithm would be more optimal than the optimal solution since in that case we could choose i_{r+1} instead of j_{r+1} and have at least an equivalent solution to the optimal or a better solution.

We can repeat this thinking on every element to prove that the algorithm yields an optimal solution.

So the locally optimal solution is in fact the global solution and so it is a greedy algorithm.

Q2. Consider a modification to the activity-selection problem in which each activity a_i has, in addition to a start and finish time, a value v_i . The objective is no longer to maximize the number of activities scheduled, but instead to maximize the total value of the activities scheduled. That is, the goal is to choose a set A of compatible activities such that $\sum_{a_k \in A} v_k$ is maximized. Give a polynomial-time algorithm for this problem.

Shai Goldberg-Kellman
Gabe Small

Q3. Recall the 0-1 knapsack problem discussed in class. A variation of this is the fractional knapsack problem, where the setup is the same, but the thief is allowed to take fractions of items instead of making an all-or-nothing (0-1) choice for each item. Prove that the fractional knapsack problem has the greedy-choice property.

We can think of the ratio of value to weight for each item.

A greedy algorithm would be to take as much of the highest value-to-weight item as possible.

Let us assume that the first r amount of items is the same between our algorithm and the optimal solution. The next bit of item in the optimal cannot have a higher value-to-weight ratio than the next bit from our algorithm as our algorithm took the highest value-to-weight item. But the algorithm cannot choose a higher value-to-weight item than the optimal solution, as then the optimal solution is not optimal. This causes the first $r+\varepsilon$ to be the same. This can be extended to the full solution.

So the locally optimal solution is in fact the global solution and so it is a greedy algorithm.

Shai Goldberg-Kellman
Gabe Small

Q4. What is an optimal Huffman code for the following set of frequencies, based on the first 8 Fibonacci numbers?

a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21

Can you generalize your answer to find the optimal code when the frequencies are the first n Fibonacci numbers?

(a:1),(b:1),(c:2),(d:3),(e:5),(f:8),(g:13),(h:21)
[(a:1)(b:1):2],(c:2),(d:3),(e:5),(f:8),(g:13),(h:21)
(d:3),{[(a:1)(b:1):2](c:2):4},(e:5),(f:8),(g:13),(h:21)
(e:5),<(d:3){[(a:1)(b:1):2](c:2):4}:7>,(f:8),(g:13),(h:21)
(f:8),[(e:5)<(d:3){[(a:1)(b:1):2](c:2):4}:7>:12],(g:13),(h:21)
(g:13),{(f:8)[(e:5)<(d:3){[(a:1)(b:1):2](c:2):4}:7>:12]:20},(h:21)
(h:21),<(g:13){(f:8)[(e:5)<(d:3){[(a:1)(b:1):2](c:2):4}:7>:12]:20}:33>
[(h:21)<(g:13){(f:8)[(e:5)<(d:3){[(a:1)(b:1):2](c:2):4}:7>:12]:20}:33>:54]

a	b	c	d	e	f	g	h
1111100	111101	11111	11110	1110	110	10	0

For every additional one add a 1 to the left

Q5. Generalize Huffman's algorithm to ternary codewords (i.e., codewords using the symbols 0, 1, and 2), and prove that it yields optimal ternary codes.

Instead of taking the top two from the priority queue, take the top three instead, put them as children of a summed node and insert it back into the priority queue.

Q6. Give a simple implementation of Prim's algorithm that runs in $O(V^2)$ time when the graph $G = (V, E)$ is represented as an adjacency matrix.

```
Prim(G, V)
Input: A VxV adjacency matrix of the graph G accessed by [][]
Output: An MST of G
0 Copy G to G {if needed to prevent changing the original}
1 Initialize MinHeap H
2 Initialize Set MST
3 for i <- 1 to V do
4     G[i][1] <- "In tree"
5 for i <- 1 to V do
6     if G[1][i] != infinity or G[1][i] != "In tree" then
7           Insert(H, (key: G[1][i], data:(1,i))
8 while not H.isEmpty() do
9     lightEdge <- ExtractMin(H)
10    MST.insert(lightEdge.data)
11    light <- lightEdge.data[2]
12    for i <- 1 to V do
13          G[i][light] <- "In tree"
14    for i <- 1 to V do
15          if G[light][i] != infinity or G[light][i] != "In tree"
then
16                Insert(H, (key: G[light][i], data:(light,i))
17 return MST
```

Q7. Suppose that all edge weights in a graph are integers in the range from 1 to $|V|$. How fast can you make Kruskal's algorithm run? What if the edge weights are integers in the range from 1 to W for some constant W ?

Q8. A bottleneck spanning tree T of an undirected graph G is a spanning tree of G whose largest edge weight is minimum over all spanning trees of G. The value of the bottleneck spanning tree is the weight of the maximum-weight edge in T.

a. Argue that a minimum spanning tree is a bottleneck spanning tree. That is, finding a bottleneck spanning tree is no harder than finding a minimum spanning tree.

b. Give a linear-time algorithm that, given a graph G and an integer b, determines whether the value of the bottleneck spanning tree is at most b.